

Р2. Игровой цикл и ввод

Цель урока: превратить неподвижную сцену в управляемую игровую сцену, где герой двигается от клавиатуры, не выходит за край окна, а программа сама повторяет шаги игры каждый кадр: прочитать ввод, обновить мир, нарисовать кадр и показать его.

Что сделать на уроке

- Запусти `lessons/lesson2/samples/01_keyboard_game_loop.py`.
- Поменяй скорость, размер или цвет героя.
- Проверь движение стрелками или клавишами `WASD`.
- Сделай так, чтобы герой оставался внутри экрана.
- Посмотри, как герой блокируется стеной.
- Исправь `lessons/lesson2/puzzles/01_hero_escapes_screen.py`.
- Сохрани свою версию игры в `team/<team_name>/`.

Какие файлы здесь есть

- `lesson2_slides.pdf` и `lesson2_slides.marp.md` — короткое визуальное объяснение для урока.
- `lesson2_guide.pdf` и `lesson2_guide.md` — эта памятка с нужными функциями и выражениями.
- `samples/01_keyboard_game_loop.py` — главный рабочий пример: игровой цикл, клавиатура, движение, границы и стена.
- `puzzles/01_hero_escapes_screen.py` — маленькая поломка: герой уходит за экран, нужно найти и исправить причину.
- `samples/` и `puzzles/` можно читать, запускать и менять для тренировки, но свою работу сохраняй в `team/<team_name>/`.

Игровой цикл и окно

Функции и выражения	Пример	Описание	Применил
<code>game.set_window_title()</code>	<code>game.set_window_title("Моя игра")</code>	Задаёт название окна.	☐
<code>game.set_window_size()</code>	<code>game.set_window_size(800, 520)</code>	Задаёт размер окна в пикселях.	☐
<code>game.open()</code>	<code>game.open()</code>	Открывает окно игры.	☐
<code>while</code>	<code>while not game.is_close_clicked():</code>	Повторяет команды, пока окно не закрыто.	☐
<code>game.is_close_clicked()</code>	<code>not game.is_close_clicked()</code>	Проверяет, нажали ли закрытие окна.	☐
<code>game.clear_canvas()</code>	<code>game.clear_canvas()</code>	Очищает экран перед новым кадром.	☐
<code>game.show_canvas()</code>	<code>game.show_canvas()</code>	Показывает готовый кадр и читает новые события.	☐
<code>game.window_size</code>	<code>game.window_size.width</code>	Даёт текущую ширину и высоту окна.	☐

Ввод

Функции и выражения	Пример	Описание	Применил
<code>game.is_key_down()</code>	<code>game.is_key_down("right")</code>	Проверяет, удерживается ли клавиша. Подходит для движения.	☐
<code>game.is_key_clicked()</code>	<code>game.is_key_clicked("space")</code>	Проверяет один короткий клик клавиши. Подходит для прыжка или меню.	☐
<code>game.is_mouse_down()</code>	<code>game.is_mouse_down("left")</code>	Проверяет, удерживается ли кнопка мыши.	☐
<code>game.is_mouse_clicked()</code>	<code>game.is_mouse_clicked("left")</code>	Проверяет один клик мыши.	☐
<code>game.get_mouse_position()</code>	<code>mouse = game.get_mouse_position()</code>	Возвращает позицию курсора как <code>Point</code> .	☐

Время

Функции и выражения	Пример	Описание	Применил
<code>game.get_delta_time()</code>	<code>dt = game.get_delta_time()</code>	Сколько секунд прошло с прошлого кадра. Нужно для плавного движения.	☐
<code>game.get_time()</code>	<code>seconds = game.get_time()</code>	Сколько секунд прошло с запуска игры.	☐
<code>game.get_fps()</code>	<code>fps = game.get_fps()</code>	Показывает, сколько кадров в секунду рисует игра.	☐
<code>speed * dt</code>	<code>hero.x += speed * dt</code>	Движение с одинаковой скоростью на разных компьютерах.	☐

Рисование

Функции и выражения	Пример	Описание	Применил
<code>game.set_fill_color()</code>	<code>game.set_fill_color("#22c55f")</code>	Выбирает цвет заливки для следующих фигур и текста.	☐
<code>game.set_stroke_color()</code>	<code>game.set_stroke_color("transparent")</code>	Выбирает цвет обводки. <code>transparent</code> убирает обводку.	☐
<code>game.draw_rectangle()</code>	<code>game.draw_rectangle(position, size)</code>	Рисует прямоугольник. Удобно для героя, стены или пола.	☐
<code>game.draw_circle()</code>	<code>game.draw_circle(Point(100, 100), 20)</code>	Рисует круг.	☐
<code>game.draw_text()</code>	<code>game.draw_text("Счёт", Point(20, 20), 24)</code>	Пишет текст на экране.	☐
<code>game.draw_image()</code>	<code>game.draw_image(hero_image, Point(100, 100), Size(64, 64))</code>	Рисует картинку из файла.	☐
<code>game.load_image()</code>	<code>hero_image = game.load_image("hero.png")</code>	Загружает картинку, чтобы потом её нарисовать.	☐

Данные игры

Функции и выражения	Пример	Описание	Применил
<code>Object</code>	<code>hero = Object(position=Point(100, 200), speed=260)</code>	Хранит данные одного объекта игры.	☐
<code>Point</code>	<code>Point(100, 200)</code>	Позиция на экране: <code>x</code> и <code>y</code> .	☐
<code>Vector</code>	<code>direction = Vector(1, 0)</code>	Направление или сдвиг.	☐
<code>Size</code>	<code>Size(48, 48)</code>	Размер: ширина и высота.	☐
Переменная	<code>speed = 260</code>	Имя для значения, которое можно менять и использовать дальше.	☐
<code>+=</code>	<code>hero.position.x += 5</code>	Увеличить значение.	☐
<code>-=</code>	<code>hero.position.y -= 5</code>	Уменьшить значение.	☐
<code>list</code>	<code>walls = [wall1, wall2]</code>	Список нескольких объектов.	☐

Условия и свои функции

Функции и выражения	Пример	Описание	Применил
<code>if</code>	<code>if</code> <code>game.is_key_down("right"):</code>	Выполняет команды, если условие верно.	☐
<code>elif</code>	<code>elif</code> <code>game.is_key_down("left"):</code>	Следующая проверка, если прошлое <code>if</code> не сработало.	☐
<code>else</code>	<code>else: speed = 0</code>	Что делать, если условия выше не сработали.	☐
<code>and</code>	<code>x > 0 and x < 800</code>	Оба условия должны быть верны.	☐
<code>or</code>	<code>key == "left" or key == "a"</code>	Достаточно одного верного условия.	☐
<code>not</code>	<code>not</code> <code>game.is_close_clicked()</code>	Переворачивает условие: “не закрыто”.	☐
<code>def</code>	<code>def update_world(world):</code>	Создаёт свою функцию, чтобы код был короче и понятнее.	☐
<code>return</code>	<code>return direction</code>	Возвращает результат из функции.	☐
<code>for</code>	<code>for wall in walls:</code>	Повторяет действие для каждого объекта в списке.	☐

Проверка границ

Функции и выражения	Пример	Описание	Применил
<code>min()</code>	<code>min(x, 800)</code>	Не даёт числу стать больше верхней границы.	☐
<code>max()</code>	<code>max(x, 0)</code>	Не даёт числу стать меньше нижней границы.	☐
<code>max() + min()</code>	<code>x = max(0, min(x, 800))</code>	Зажимает число внутри диапазона.	☐
Проверка прямоугольников	<code>hero.x < wall.x + wall.width</code>	Помогает понять, касается ли герой стены.	☐